National University of computer and emerging sciences

SUM Technologies

UniVerse

Event & Society Management System

Software Requirements & Design Document

Deliverable: Final Report

Course	Software Design and Analysis
Submitted To	Dr. Afia Zafar
Team	SUM Technologies
Members	Ubaid Ashraf – 24i-0841 Muhammad Shahzaib Khan – 24i-0741 Mitul Dial – 24i-0791

Table of Content

1. Introduction

1.1 Purpose

This document is the Software Requirements and Design Specification for **UniVerse**, an all-in-one event and society management desktop application developed for university environments. It defines the functional and non-functional requirements, architecture, and design artefacts governing the development of UniVerse.

1.2 Product Scope

UniVerse is a JavaFX desktop application backed by Microsoft SQL Server. It acts as a single, unified hub that connects three core groups students, societies, and sponsors while giving administrators full visibility and control over everything that happens on the platform.

The scope covers student and society self-registration, event creation and approval, a shared calendar for conflict detection, an announcements tab, a sponsorship-proposal workflow, and an admin dashboard. Features such as mobile access, online payment, and third-party platform integration are explicitly out of scope.

1.3 Project Title

UniVerse: An Event and Society Management System

The name reflects the ambition of creating one unified 'universe' for campus life where every student, society, and sponsor can find everything they need without switching between WhatsApp groups, Instagram pages, or physical notice boards.

1.4 Objectives

The main objectives of this desktop based application are to:

- Build a centralized desktop platform for managing all university society events.
- Give students a single place to view event details, register, track their registrations, and read society announcements.
- Enable societies to create events, post announcements, and check the shared calendar so they never double-book a venue or date.
- Provide sponsors with a structured channel to discover upcoming events and submit sponsorship proposals directly to a society.
- Implement a role-based access system so each user type sees only the features relevant to them nothing more, nothing less.

- Send automated notifications when a new event is approved, an announcement posted, or a sponsored event is completed.
- Maintain a live admin dashboard showing pending event approvals, platform statistics, and recent activity.

1.5 Problem Statement

Campus societies organize a wide range of events hackathons, sports competitions, cultural fests, recruitment drives, and more. The current reality is that all communication about these events is fragmented: WhatsApp messages get buried, Instagram posts are missed, physical posters are torn down, and class-to-class marketing reaches only a fraction of the student body. The direct consequence is low turnout, missed opportunities for students, and wasted effort from society organizers.

The problem is made worse by the absence of any shared scheduling mechanism. Without visibility into each other's calendars, societies routinely clash on dates and venues. Sponsors, meanwhile, have no structured channel for finding events to back, so both sides lose out on partnerships that could otherwise happen.

UniVerse solves all of these problems at once. By centralizing information, automating conflict detection, and providing a formal sponsorship workflow, the platform removes friction for every stakeholder and turns a fragmented, informal system into something organized, transparent, and reliable.

2.Overall Description

2.1 Product Perspective

UniVerse is a standalone desktop application. It does not extend any existing system it is built from scratch to replace the informal, frustrating communication channels currently used by university societies. The system is structured in three logical layers.

- **Presentation layer:** JavaFX FXML views and controllers. Each user type gets a role-appropriate dashboard with tabs that are shown or hidden depending on the logged-in role. The UI is fully separated from business logic controllers call service classes, never the database directly.
- **Service layer:** Java service classes (EventService, RegistrationService, NotificationService, SocietyService, SponsorshipService, UserService) that contain all business rules. This layer is shared across all roles; the role filter sits in the presentation layer.

- **Data layer:** Microsoft SQL Server accessed through a JDBC connection. The DBConnection class is a singleton, so the application opens exactly one connection per session and reuses it throughout. All database queries use PreparedStatement to prevent SQL injection.

2.2 Product Functions

- **Authentication and registration:** Students, societies, and sponsors can create accounts from the landing page. Admin and sponsor accounts are seeded directly in the database. On sign-in the system checks credentials via a parameterized SQL query and rejects the login with an error message if they do not match. Duplicate email registration is blocked before the INSERT is issued.
- **Role-based dashboard:** Once logged in, the user sees a dashboard whose tabs and action buttons are controlled by their role (Student, Society, Sponsor, Admin). A Session singleton holds the logged-in User object for the lifetime of the session.
- **Event creation and approval workflow:** A society creates an event using a form dialog. The event is saved with status 'Pending'. The admin sees it on the Pending Approvals list and can approve or reject it. Approval triggers a global notification broadcast to all users.
- **Events feed and calendar:** All approved events appear in a scrollable feed with date, title, venue, society name, deadline, and fee. An interactive calendar marks days that have events. Clicking a day shows the event on that date.
- **Student registration:** A student can register for any open, approved event with available seats. The system checks for duplicate registration and seat availability before writing the record. Confirmed registrations appear in the student's sidebar.
- **Announcements and notifications:** All users see an Announcements tab that pulls global notifications and role-specific notifications from the Notification table ordered by date. Events that have passed automatically trigger completion notifications the next time a user logs in.
- **Sponsorship workflow:** A sponsor browses approved events, selects one, writes a proposal message, and submits it. The owning society sees the incoming deal and can accept or reject it. Accepted deals give the sponsor access to live event statistics such as registered students and estimated revenue.
- **Admin panel:** The admin sees pending event approvals, live platform statistics (total events, students, active societies, sponsors), and the five most recent admin-targeted notifications.

- **Society dashboard:** A society user sees a list of all events they have created with status badges, seat counts, and an expandable registration list per event. They can confirm or cancel individual registrations. Incoming sponsorship proposals are shown in a separate section below.

2.3 List of Use Cases

Use Case Name	Primary Actor
Register Account	Student / Society / Sponsor
View Events	Student
Register for an Event	Student
Post Announcement	Society Representative
Create Event	Society Representative
View Shared Calendar	Society Representative
View Event Registrations	Society Representative
View Sponsorship Opportunities	Sponsor
Contact Society	Sponsor
Approve / Reject Events	Admin
Manage Societies	Admin

2.4 Extended Use Cases

UC-04: Register for an Event

Field	Description
Scope	Student registers for an approved, open event.
Primary Actor	Student
Stakeholders	Student, Society Representative, Admin
Precondition	Student is logged in. Event exists, is Approved, has available seats, and registration deadline has not passed.

Postcondition	EventRegistration row inserted (status Pending); seat count updated; student sidebar refreshed.
Main Success Scenario	1. Student navigates to Events tab. 2. System displays all approved events with status (Register /Registered / Seats Full / Closed). 3. Student clicks Register on a chosen event. 4. System calls RegistrationService.isAlreadyRegistered() returnsfalse. 5. System calls countConfirmedRegistrations() count is belowmaxSeats. 6. System inserts EventRegistration record and refreshes the buttonto 'Pending'. 7. Student sidebar is refreshed to show the new registration.
Alternate Flows	Already registered: button shows 'Registered' and is disabled. Seats full: button shows 'Seats Full' and is disabled. Deadline passed or event over: button shows 'Closed (Past)' and is disabled.

UC-06: Create Event

Field	**Description**
Scope	Society creates a new event and submits it for approval.
Primary Actor	Society Representative
Stakeholders	Society Representative, Admin, Students
Precondition	Society user is logged in. Dashboard shows '+ Add Event' button.
Postcondition	Event saved with status 'Pending'. Admin notified. Event visible on Society's own dashboard.

Main Success Scenario	1. Society clicks '+ Add Event' in the welcome banner. 2. System displays a scrollable form dialog (title, description, date,time, venue, max seats, fee, deadline). 3. Society fills in all fields and clicks Create. 4. System calls EventService.checkVenueConflict() no conflict.5. EventService.createEvent() runs a parameterised INSERT with status 'Pending'. 6. A success toast confirms submission. 7. Event feed is refreshed.
Alternate Flows	Invalid input (e.g. non-numeric seats): system shows 'Invalid input' error toast. Venue conflict detected: system surfaces the conflict before saving.

UC-10: Contact Society (Sponsorship Proposal)

Field	Description
Scope	Sponsor submits a proposal to sponsor an event.
Primary Actor	Sponsor
Stakeholders	Sponsor, Society Representative, Admin
Precondition	Sponsor is logged in. At least one approved event exists.
Postcondition	SponsorshipDeal record inserted (status Pending). Deal visible to the owning society. Button on event changes to 'Applied'.
Main Success Scenario	1. Sponsor navigates to Sponsors tab, sees list of approved events. 2. Sponsor clicks 'Send Proposal' on a chosen event. 3. System shows a text-input dialog for the proposal message. 4. Sponsor writes the message and confirms. 5. SponsorshipService.submitProposal() inserts the deal. 6. The event's button changes to 'Applied' and is disabled. 7. Society sees the incoming deal on their dashboard.
Alternate Flows	Sponsor already applied to this event: button already shows 'Applied' on load.

2.5 Use Case Diagram

The diagram below shows the actors and their interactions with the UniVerse system. The four actors are Student, Society Representative, Sponsor, and Admin.

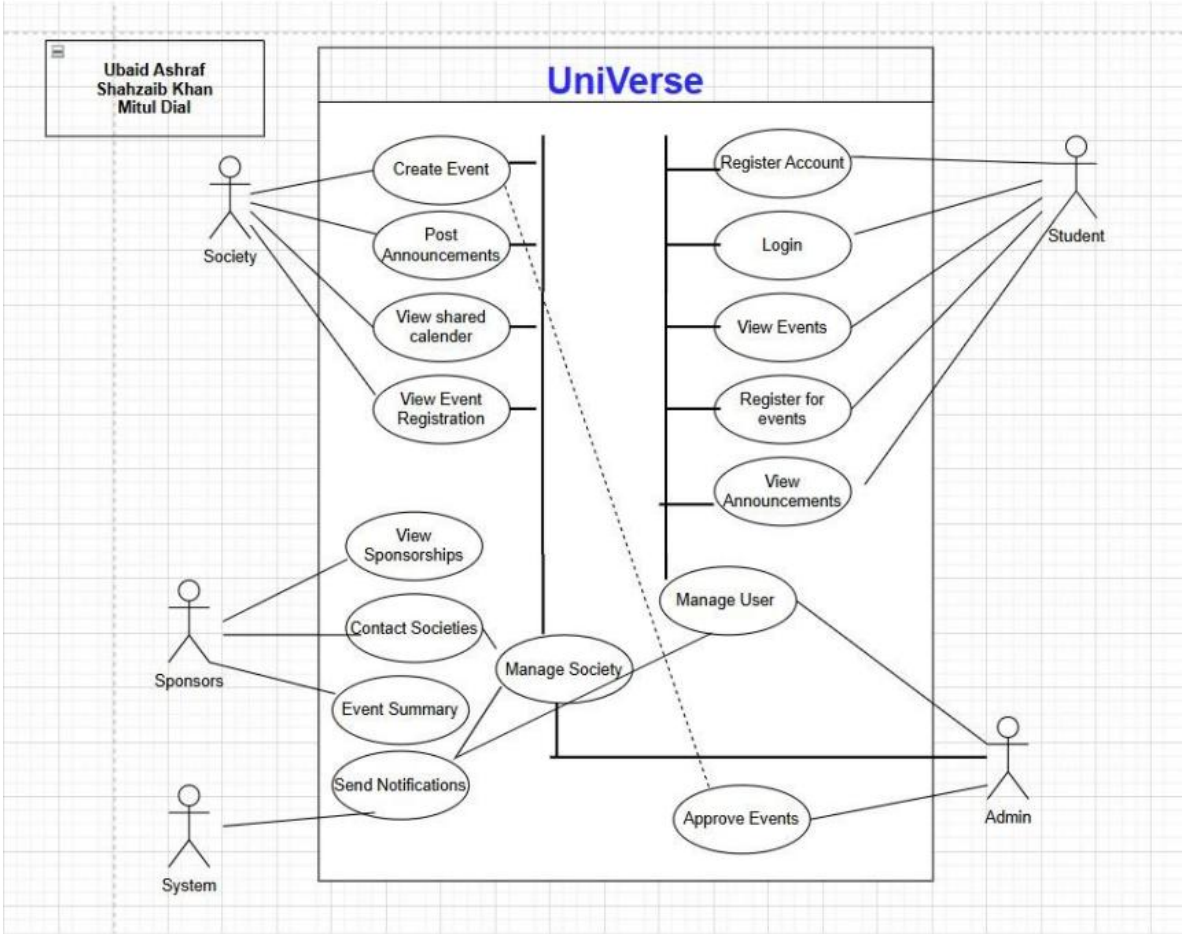


Figure 2.1 – UniVerse Use Case Diagram

3. Non-Functional Requirements

3.1 Performance Requirements

- The events feed must load all approved events and render the list within seconds on a standard university desktop connected to a local SQL Server instance.
- Credential verification on login must complete within seconds under normal database load.
- Venue conflict detection (checkVenueConflict) must respond before the event form is submitted, giving the user instant feedback.
- The admin pending-approvals list must refresh in under 1 second after an approve or reject action the current implementation removes the row with a 200 ms fade animation and refreshes the list immediately.
- Notification queries are executed at login and on every tab switch; each must complete within seconds.

3.2 Safety Requirements

- All databases write operations (INSERT, UPDATE) use Prepared Statement, ensuring atomicity at the JDBC level. A failed operation logs the SQL Exception without corrupting partial data.
- Destructive actions cancelling a registration, rejecting an event, or suspending a society are triggered through explicit button clicks and are immediately reflected in the database with a matching status update, so the state is never ambiguous.
- The DB Connection singleton ensures that only one connection object is active per application session, preventing connection leaks.
- The Session singleton is cleared when the user navigates back to the landing page, ensuring no residual user data persists between logins on the same machine.

3.3 Security Requirements

- All SQL queries use parameterized prepared statement calls, eliminating SQL injection as an attack vector.
- Role-based access control is enforced at the controller level: tabs and action buttons are made visible or invisible depending on the runtime type of the Session user (instance of checks for Student, Society, Sponsor, Admin). A user who somehow navigates to a restricted tab sees no data because the populate methods gate on the session role.

- Admin and sponsor accounts cannot be self-registered through the application. They must be inserted directly into the database by a database administrator, limiting privilege escalation.
- Email uniqueness is verified before any registration INSERT is executed (emailExistsquery), preventing account duplication.
- The JDBC connection string uses Integrated Security (Windows Authentication) so no database password is stored in the application source code.

3.4 Software Quality Attributes

Attribute	How it is achieved in UniVerse
Usability	Role-specific dashboards surface only the controls a user needs. Entrance animations, toast notifications, and hover effects give continuous feedback. All forms use validation with descriptive error toasts.
Maintainability	Code is split into model, service, controller, and db packages. Services are independent of the UI; swapping JavaFX for another frontend would not require touching Event Service or any other service class. GRASP patterns (Information Expert, Creator, Controller, Pure Fabrication) are applied throughout.
Reliability	All service methods wrap database operations in try-catch blocks and print stack traces for diagnosis without crashing the UI. The completed-events notification check runs automatically on every dashboard load, keeping data consistent.
Scalability	The architecture supports multiple universities by extending the University table and scoping society and event queries by universityID. The service layer is stateless, so adding more clients (or eventually a server) does not require architectural changes.
Testability	Business logic lives entirely in service classes with no JavaFX dependency, making unit testing straightforward with JUnit and a test database connection.
Portability	The application targets Java 11+ and JavaFX 11+, both of which run on Windows, macOS, and Linux. Only the JDBC driver DLL (.dll) is Windows-specific; a cross-platform driver can replace it without any code changes.

3.5 Business Rules

- An event may only appear on the public events feed after it has been approved by an Admin. Events in Pending or Rejected state are visible only to the society that created them.
- A student may register for the same event only once. The system enforces this with an isAlreadyRegistered query before any INSERT.
- Registration closes automatically when the regDeadline date has passed or the event date has passed no manual intervention is needed.
- A society representative can only create, view, and manage events that belong to their own societyID. They cannot see or modify another society's events.
- Admin and sponsor accounts are managed at the database level and cannot register or update their own credentials through the application.
- A sponsor may send only one active proposal per event. If a proposal exists (status not Rejected), the 'Send Proposal' button is replaced with 'Applied' and is disabled.
- When an event is approved, a GLOBAL notification is broadcast to all users and ADMIN notification is logged for audit.

3.6 Operating Environment

- **Hardware:** Any standard university desktop or laptop.
- **Runtime:** Java 11 or higher with JavaFX 11 or higher on the classpath.
- **Database:** Microsoft SQL Server (local or network instance) with the universe_db database schema deployed.
- **JDBC Driver:** mssql-jdbc 12.6.1 (included in the project via the native .dll bundled in the repo root).
- **Build Tool:** Maven (pom.xml present). IntelliJ IDEA is the recommended IDE.
- **Network:** The application connects to SQL Server on localhost by default. A network SQL Server instance can be used by updating the JDBC URL in DBConnection.

3.7 User Interfaces

UniVerse has two primary screens: a landing page and a dashboard. Both are defined in FXML files and controlled by LandingController and DashboardController respectively.

- **Landing page:** The full-screen animated page with a particle-network background. It presents a Sign In button (opens a modal dialog) and a Get Started button (opens a role-chooser then a registration form dialog). Navigation anchors scroll to Features, How It

Works, and About sections. Live statistics (universities, societies, students, events) are pulled from the database at load time.

- **Dashboard:** It is a tab-based layout. The top bar shows the application name and a user-avatar button (displays a name/email/role popup). The tab bar contains Events, Societies, Announcements, Sponsors (visible to Sponsor and Admin only), and Admin (visible to Admin only). Each tab panel is a scrollable content area that reloads live data on every tab switch.
- **Form dialogs:** All data entry (login, registration, event creation, sponsorship proposal) uses ThemedDialog modal overlays that slide over the current screen. Validation errors are shown as toast notifications that auto-dismiss.
- **Consistency:** Fonts, spacing, and control styles are defined in CSS stylesheets applied globally. Cards, badges, and buttons follow a consistent visual language across all panels.

4. Domain Model

The Domain Model shows the key entities in the UniVerse problem domain and how they relate to each other. The main entities are **University**, **Admin**, **Student**, **Society**, **Event**, **EventRegistration**, **Announcement**, **Sponsor**, **SponsorshipDeal**, **SharedCalendar**, **Notification**, and **EventSummary**.

A University hosts one or more Societies. Each Society creates zero or more Events and posts Announcements. Students make EventRegistrations for approved Events. Sponsors initiate SponsorshipDeals against Events. The Admin approves Events and manages Societies. The SharedCalendar tracks all approved Event dates to support conflict detection.

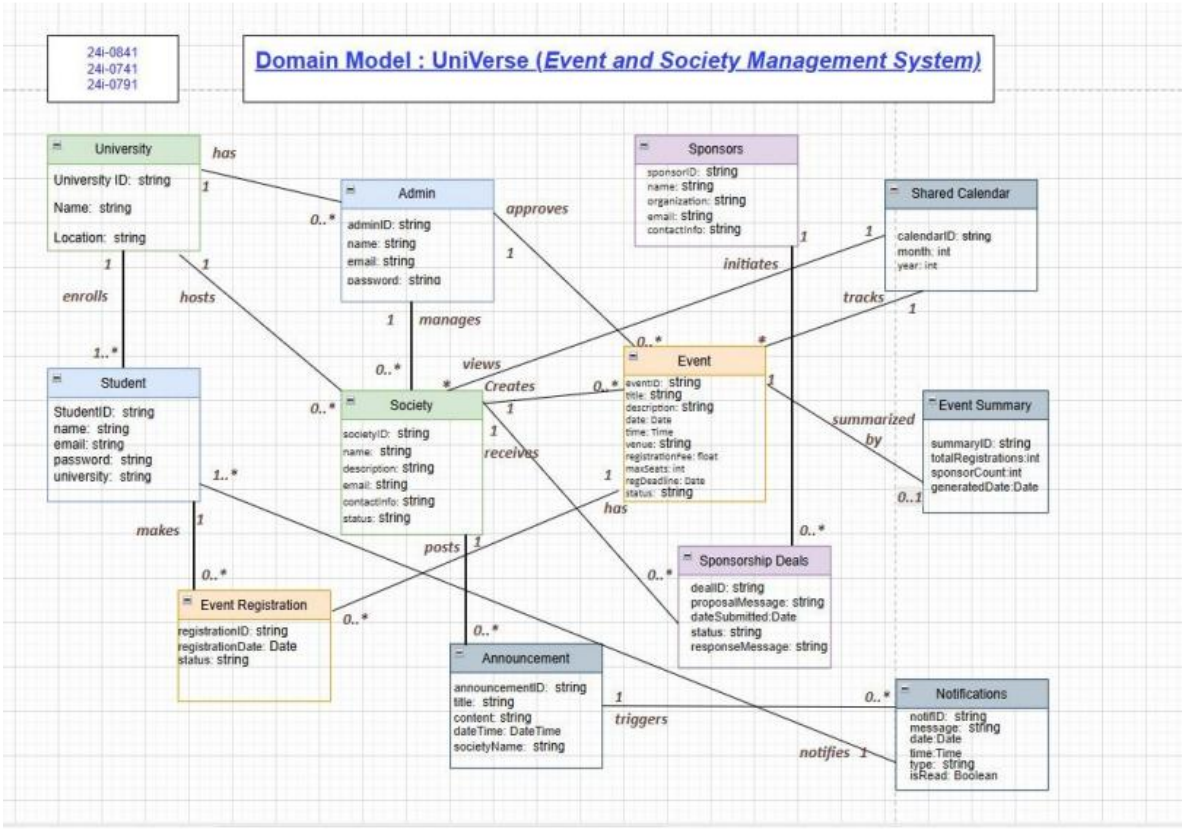


Figure 4.1 – UniVerse Domain Model

5. System Sequence Diagrams (SSDs)

System Sequence Diagrams show the messages exchanged between an actor and the system boundary for each use case. The system is treated as a black box internal components are not shown.

5.1 Register Account

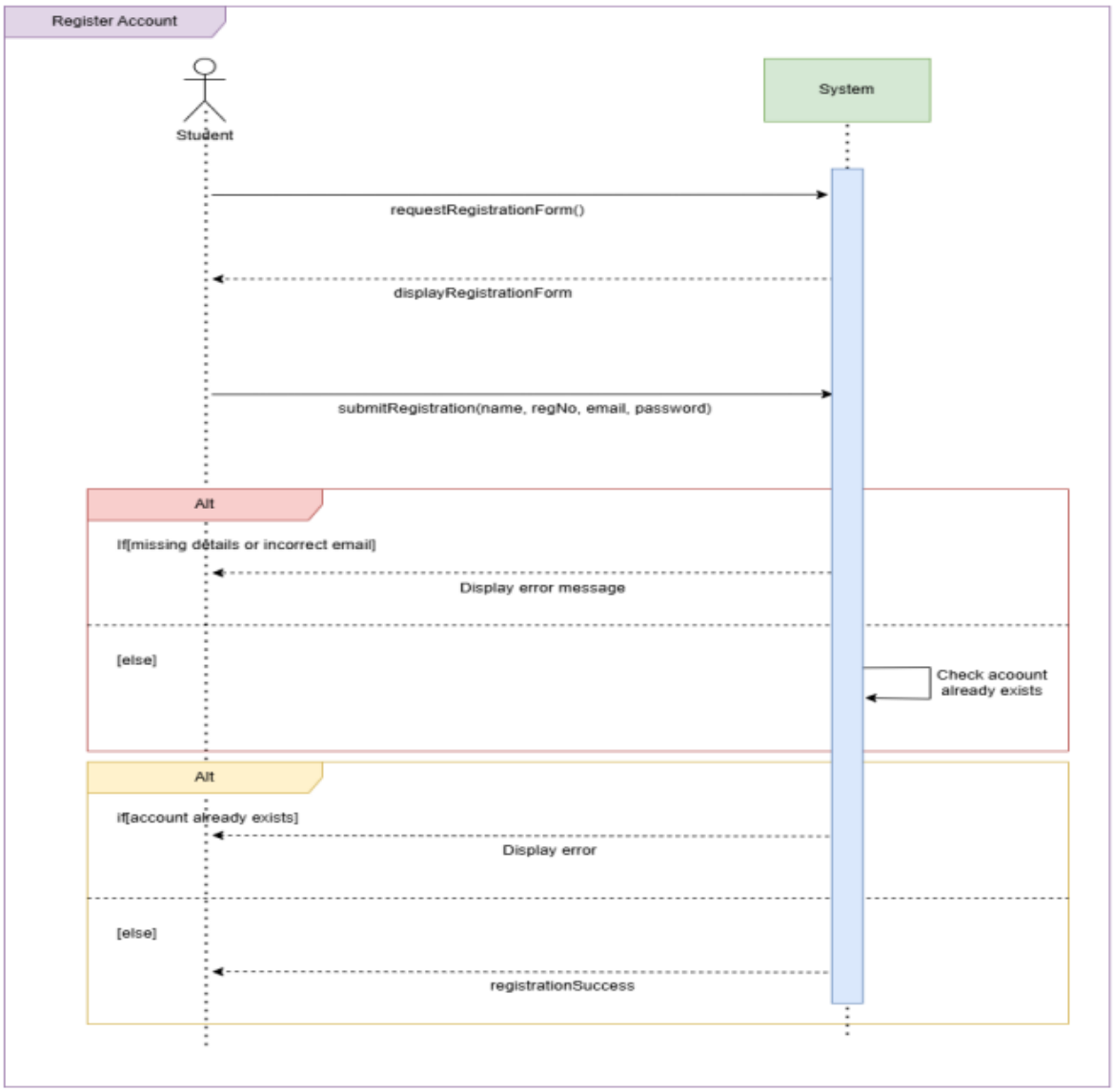


Figure 5.1 – System Sequence Diagram: Register Account

5.2 View Events

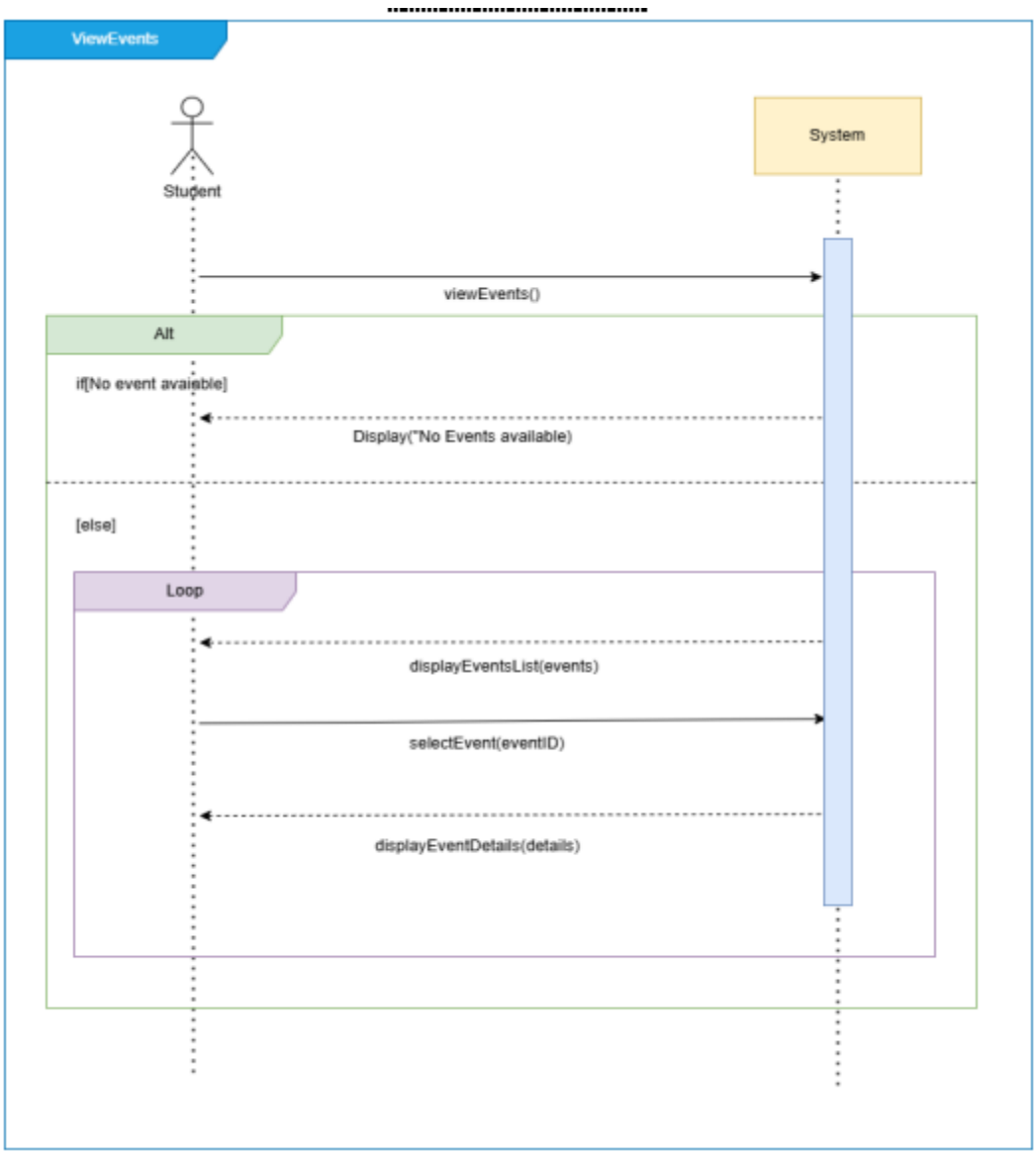


Figure 5.2 – System Sequence Diagram: View Events

5.4 Post Announcement

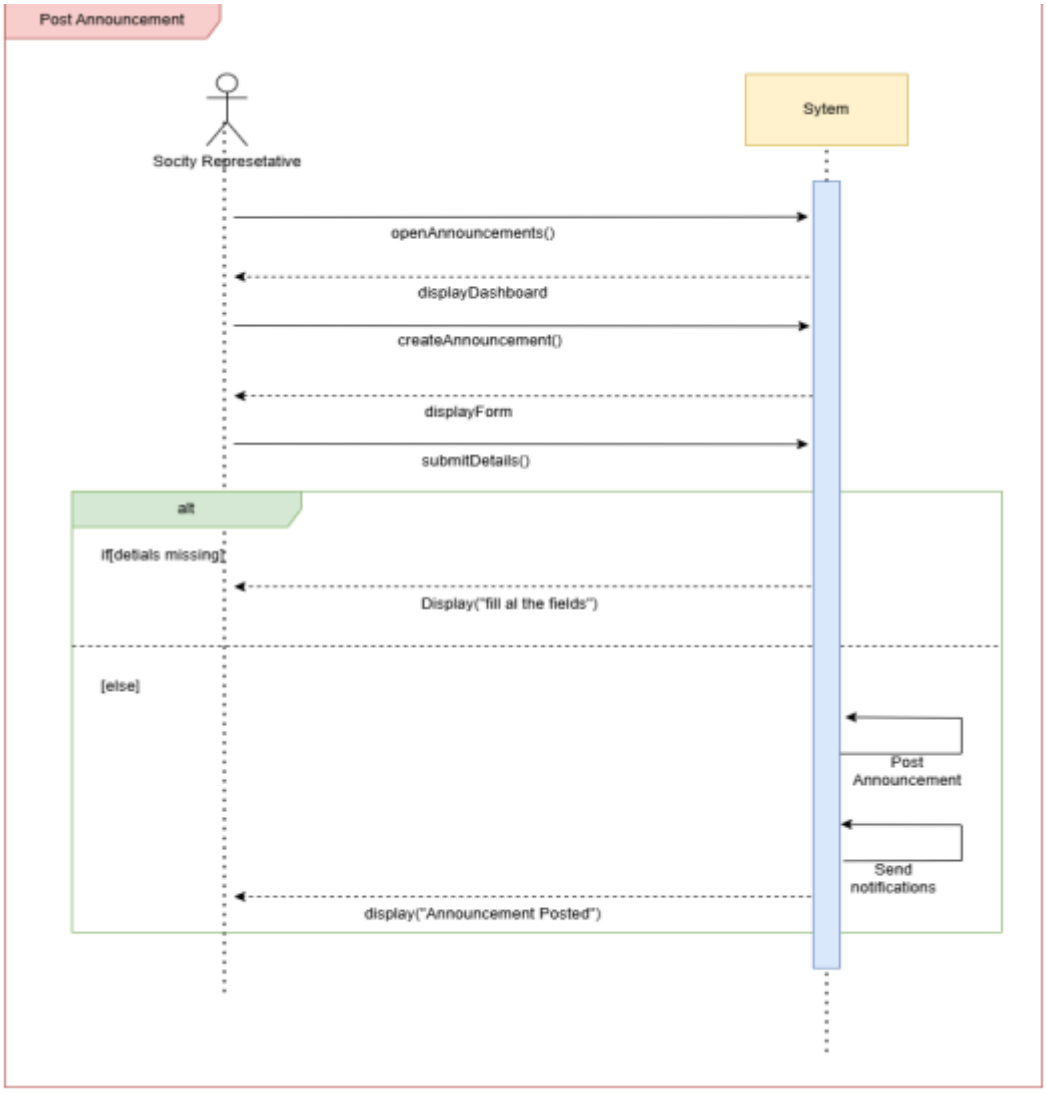


Figure 5.4 – System Sequence Diagram: Post Announcement

5.5 View Shared Calendar

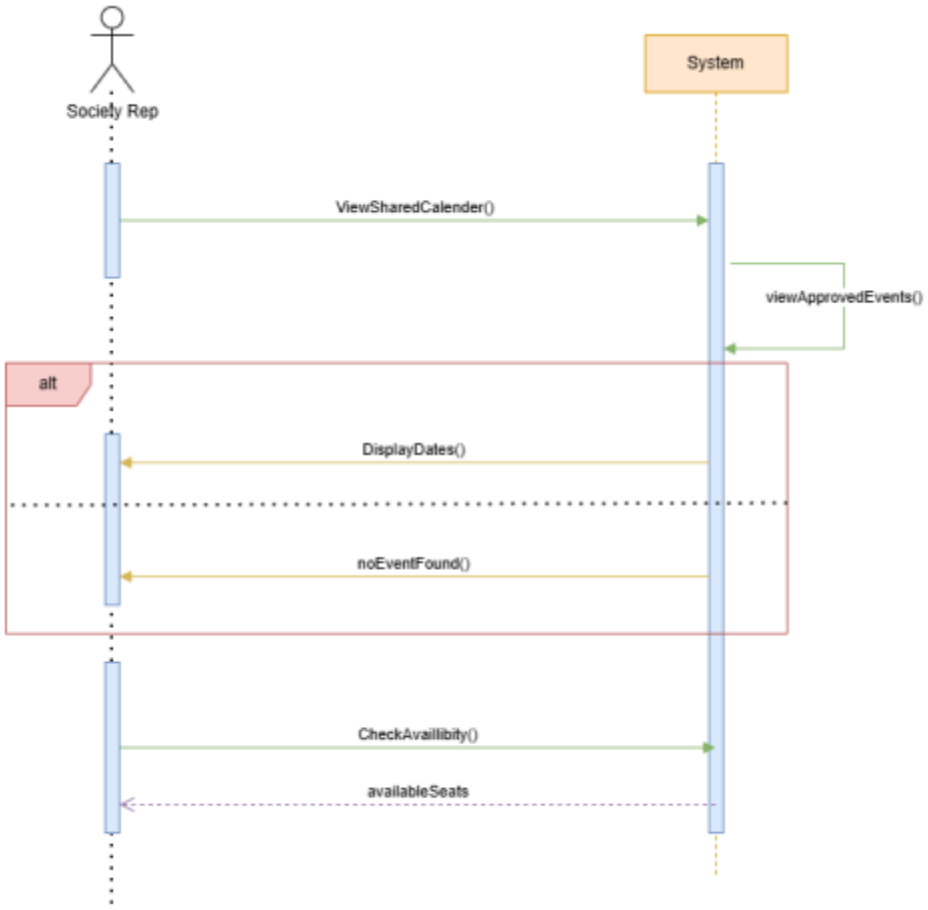


Figure 5.5 – System Sequence Diagram: View Shared Calendar

5.6 View

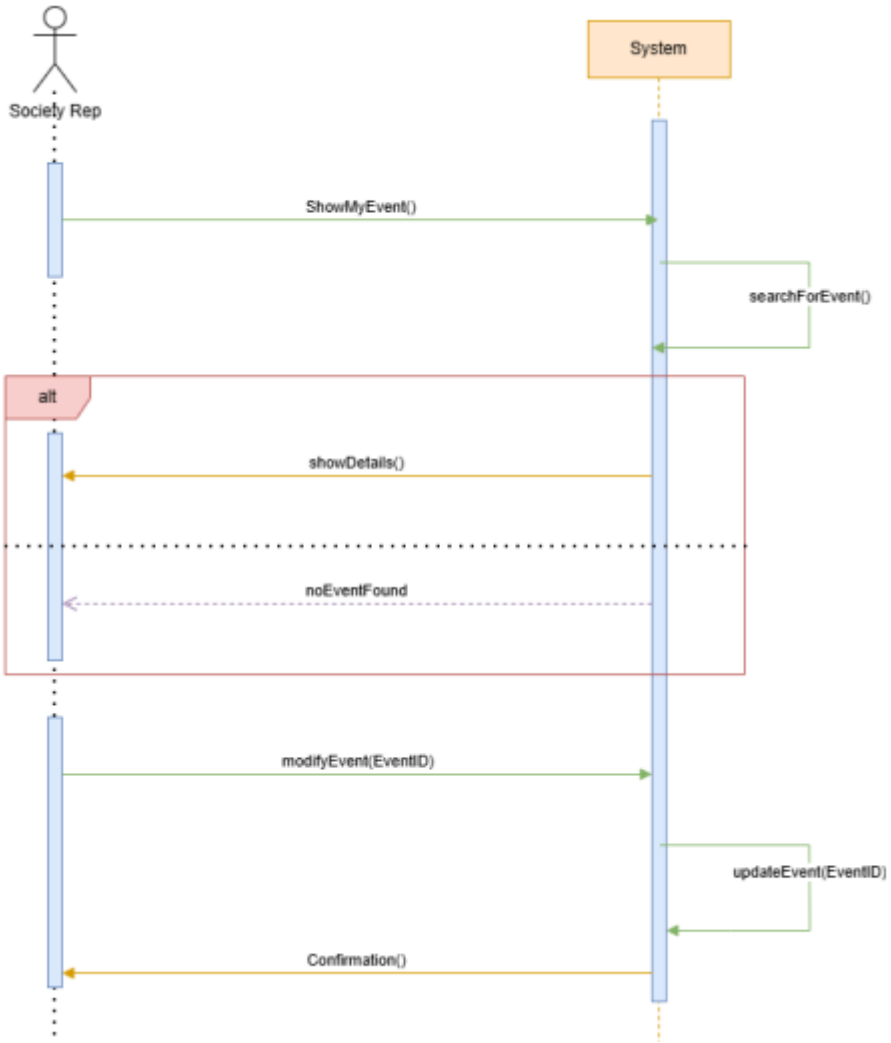


Figure 5.6 – System Sequence Diagram: View Event Registrations

5.7 View Sponsorship Opportunities

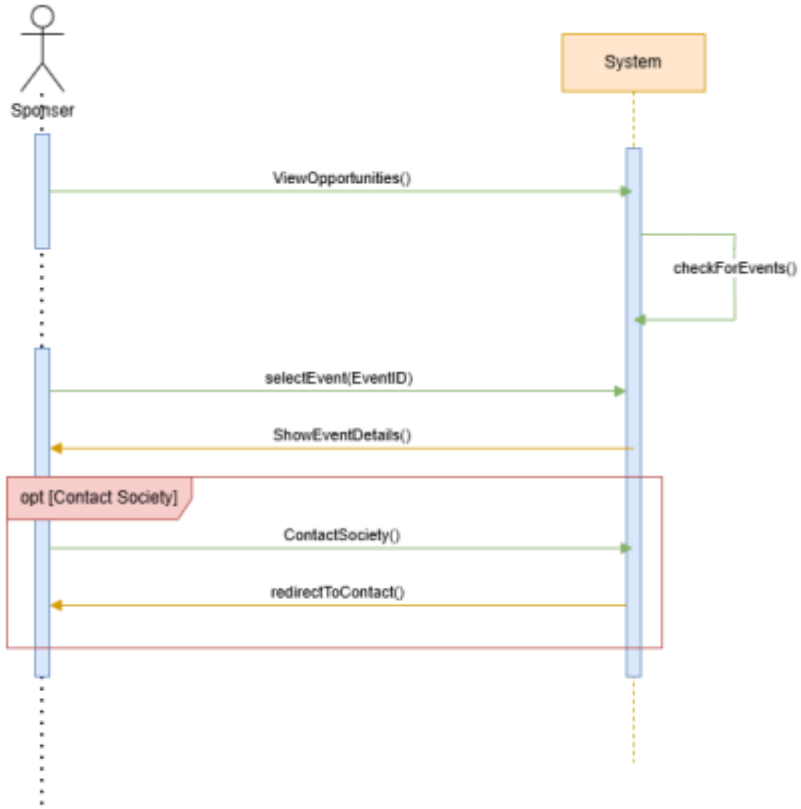


Figure 5.7 – System Sequence Diagram: View Sponsorship Opportunities

5.8 Approve Events

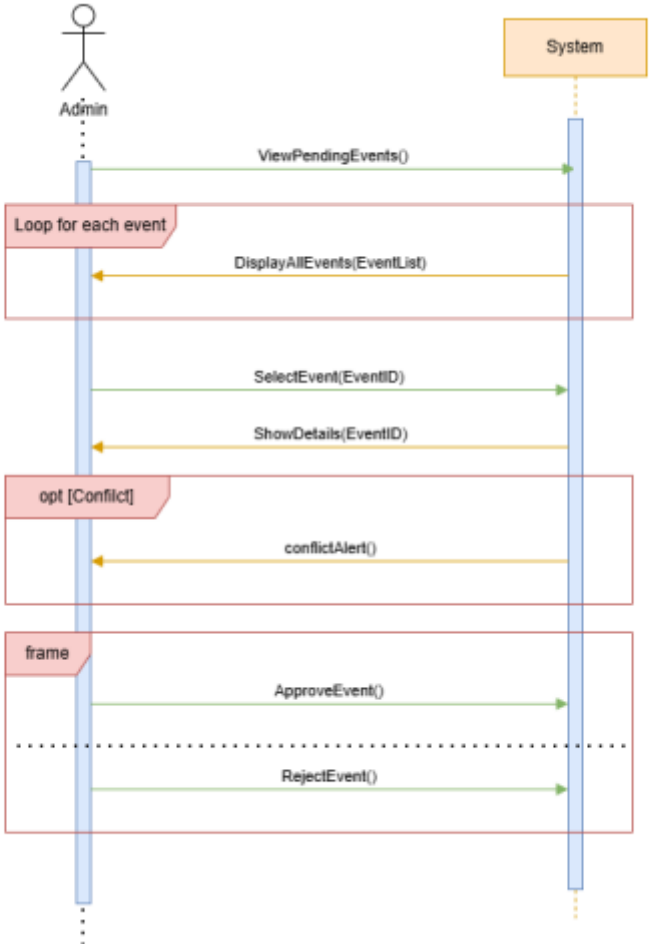


Figure 5.8 – System Sequence Diagram: Approve Events

5.9 Register for an Event

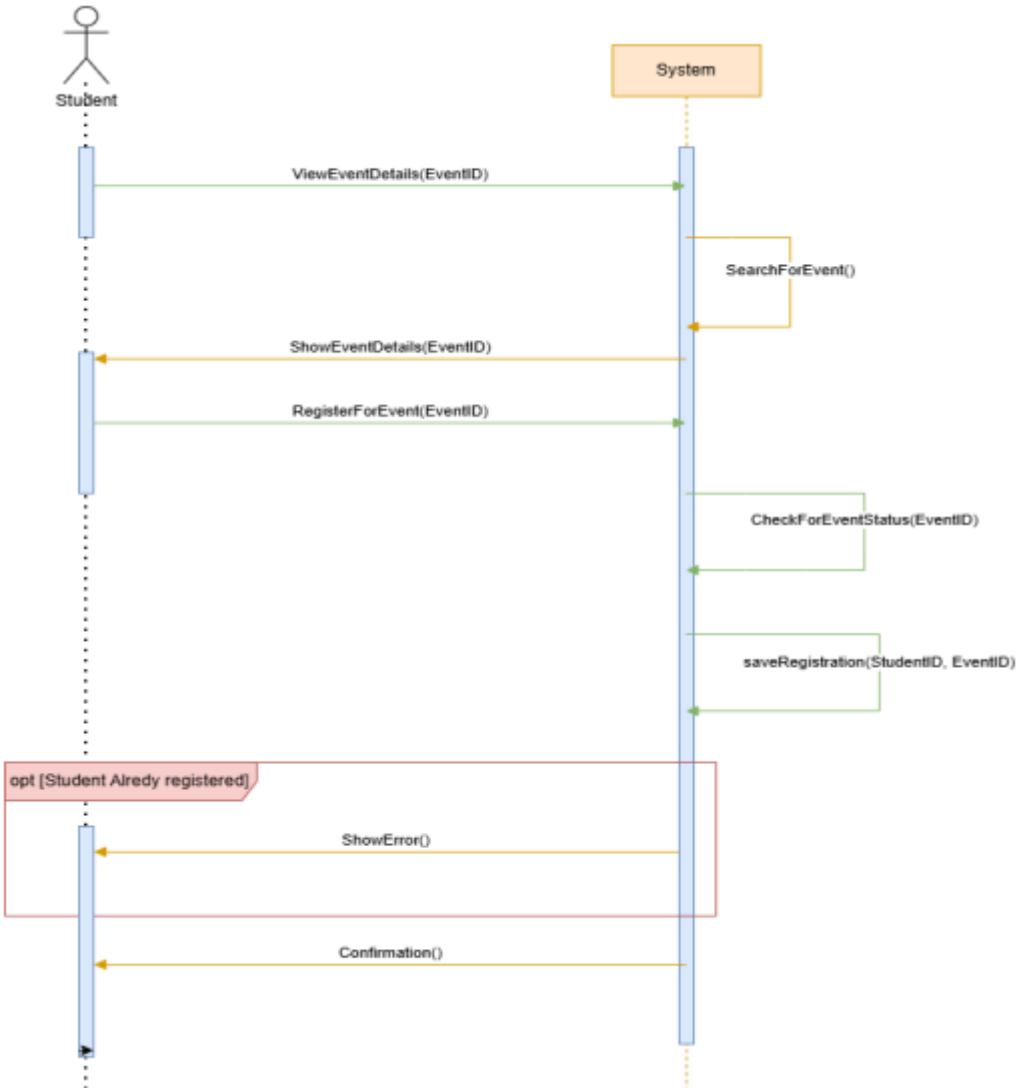


Figure 5.9 – System Sequence Diagram: Register for an Event

5.10 Create Event

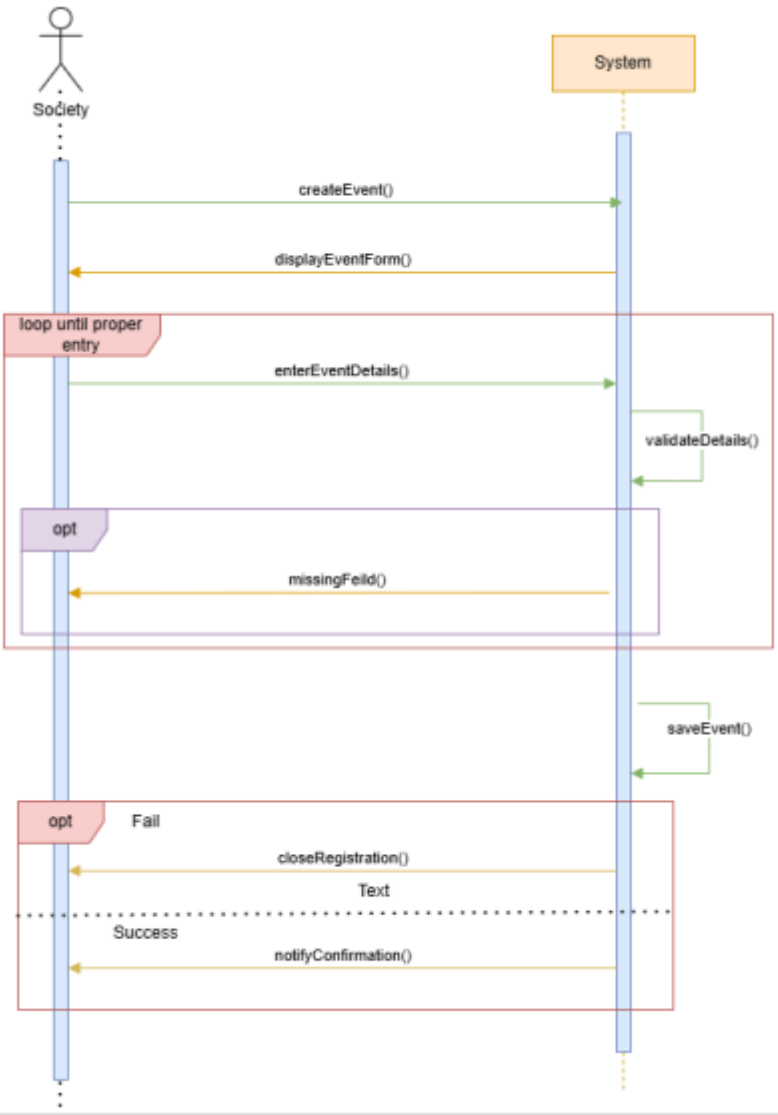


Figure 5.10 – System Sequence Diagram: Create Event

5.11 Contact Society

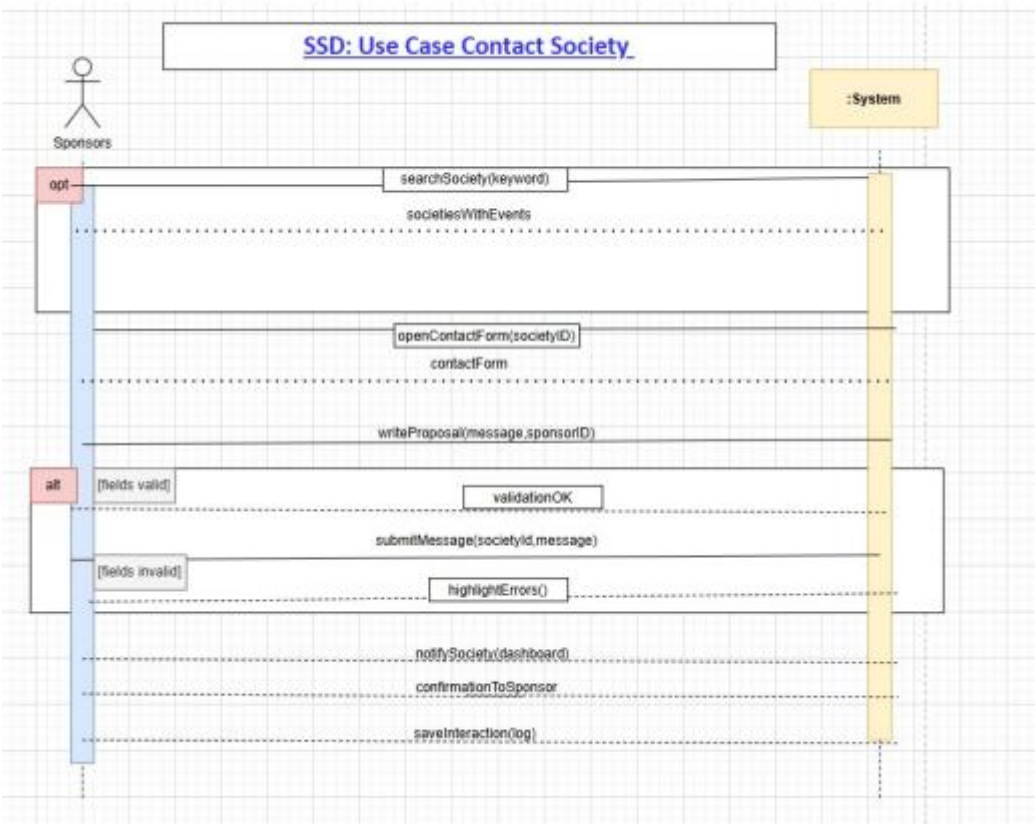


Figure 5.11 – System Sequence Diagram: Contact Society

5.12 Manage Societies

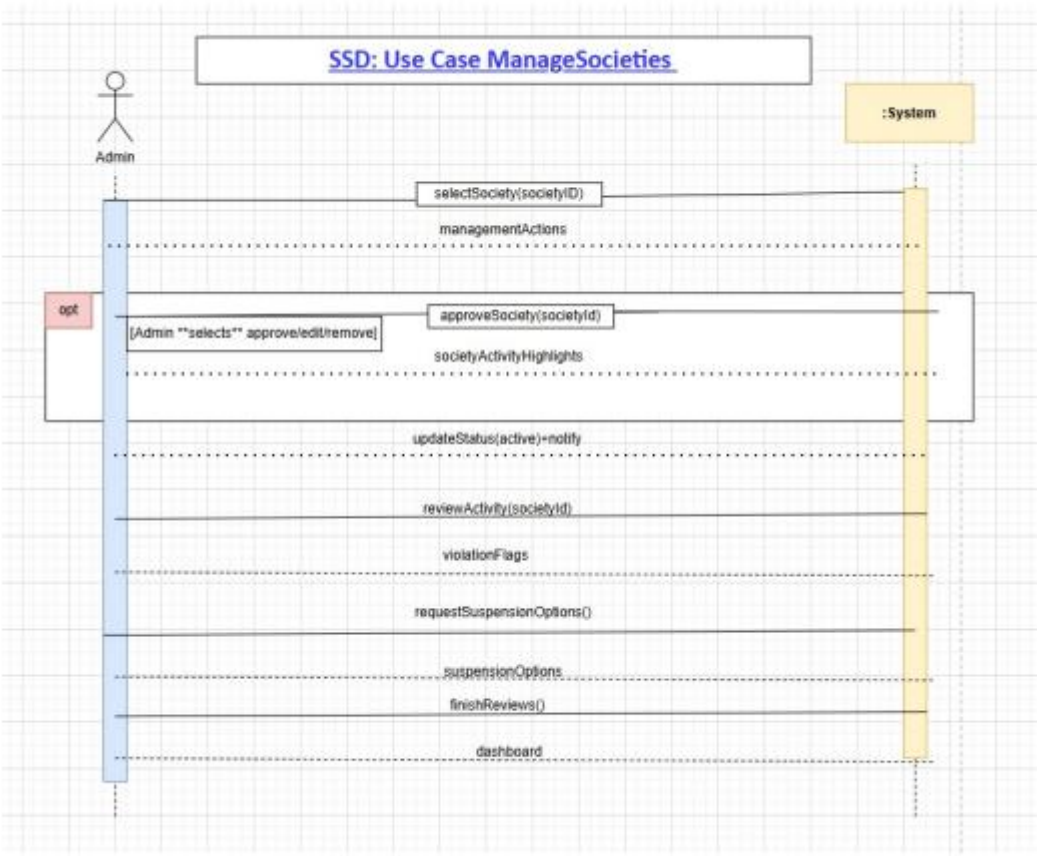


Figure 5.12 – System Sequence Diagram: Manage Societies

6. Sequence Diagrams (SDs)

Sequence Diagrams show how internal classes and objects collaborate to carry out each use case. Controllers call service methods; services interact with the database through prepared statements.

6.1 Register Account

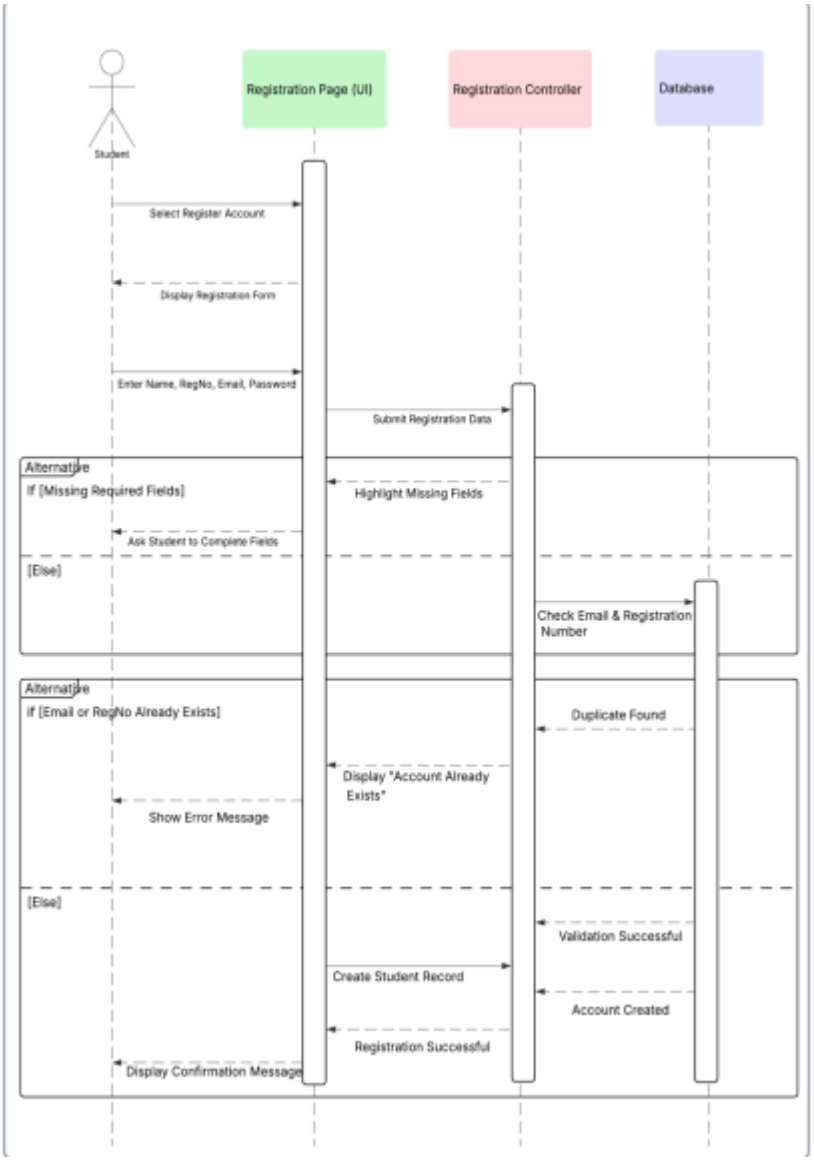


Figure 6.1 – Sequence Diagram: Register Account

6.2 Login

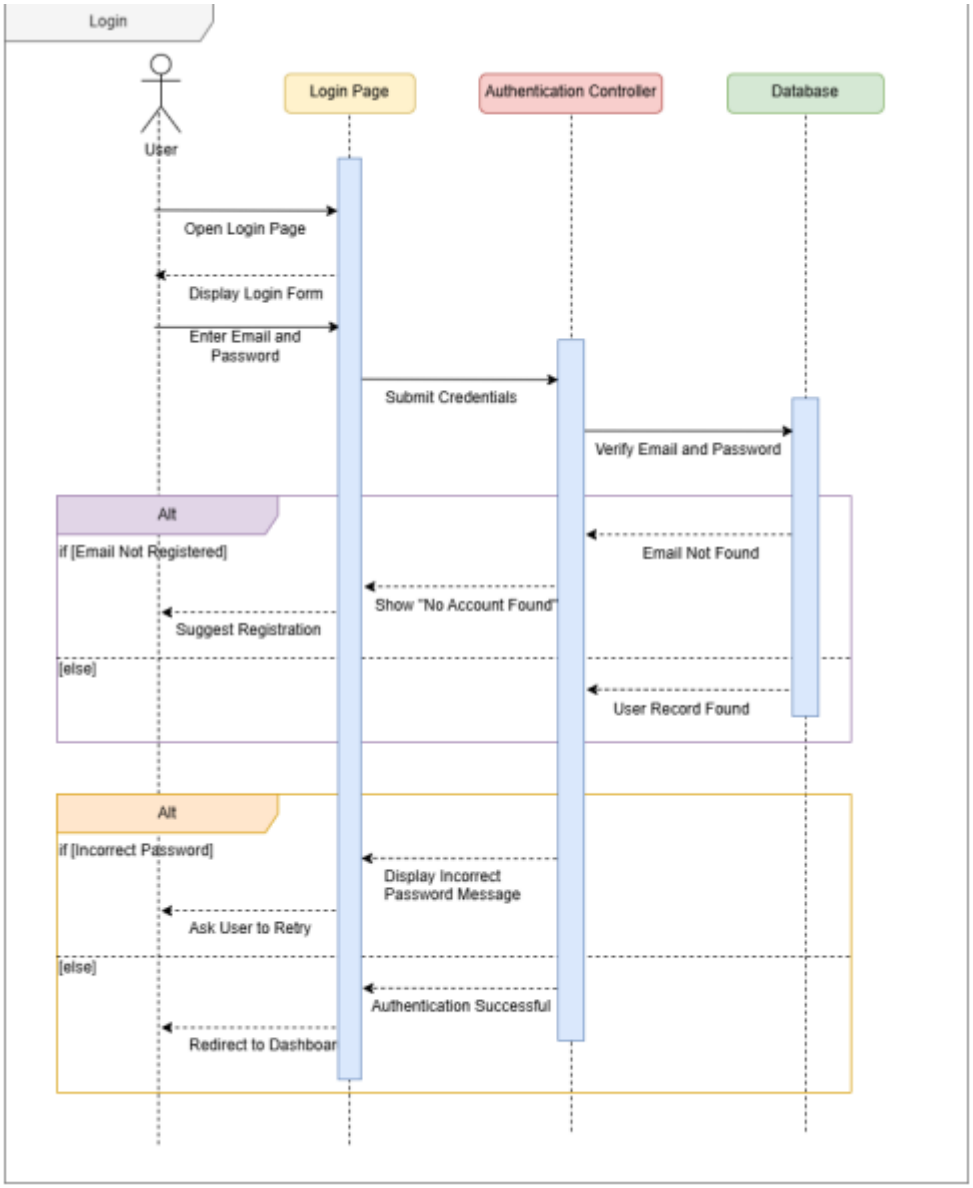


Figure 6.2 – Sequence Diagram: Login

6.3 View Events

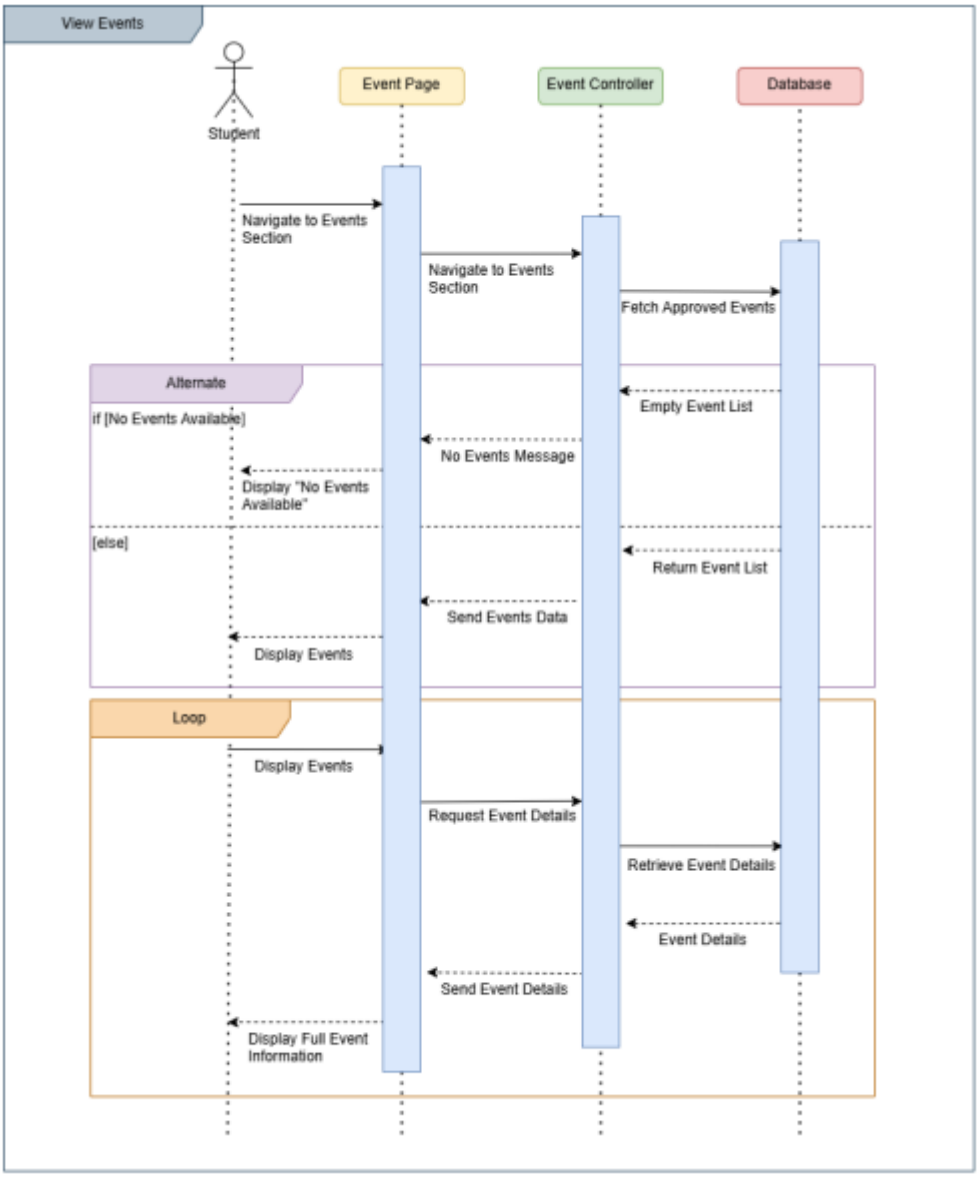


Figure 6.3 – Sequence Diagram: View Events

6.4 Post Announcement

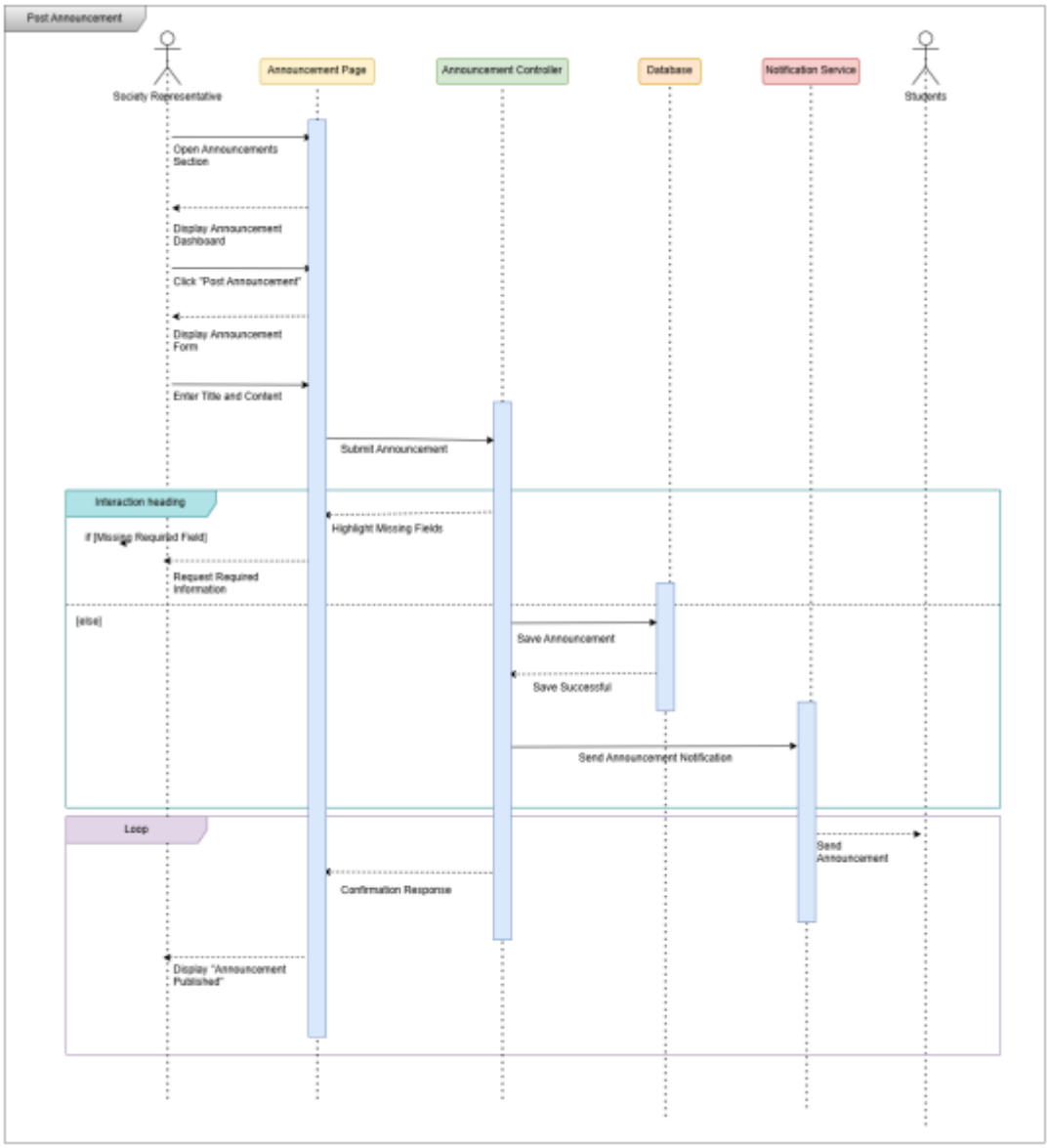


Figure 6.4 – Sequence Diagram: Post Announcement

6.5 View Shared Calendar

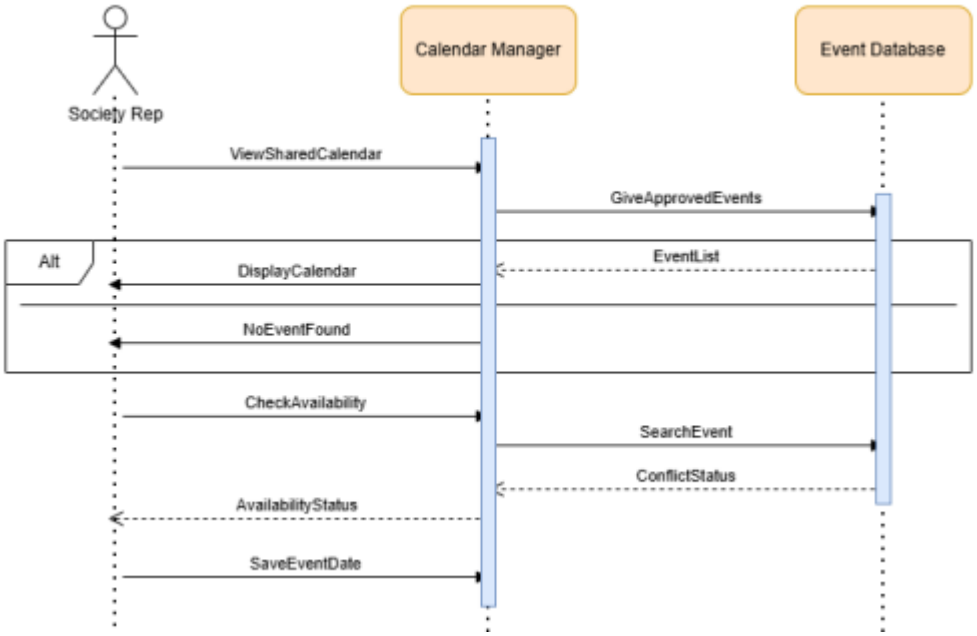


Figure 6.5 – Sequence Diagram: View Shared Calendar

6.6 View Event Registrations

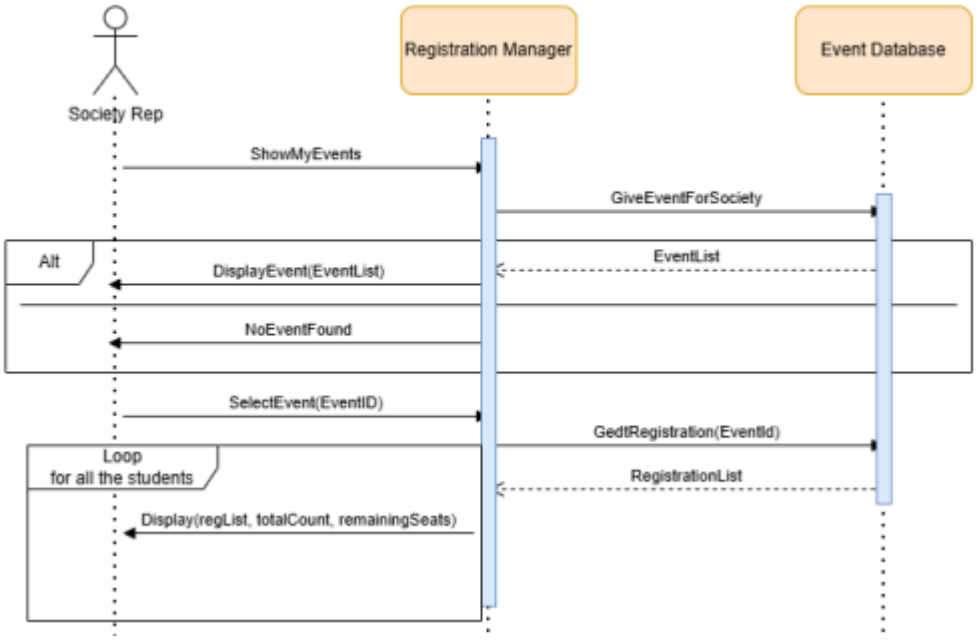


Figure 6.6 – Sequence Diagram: View Event Registrations

6.7 View Sponsorship Opportunities

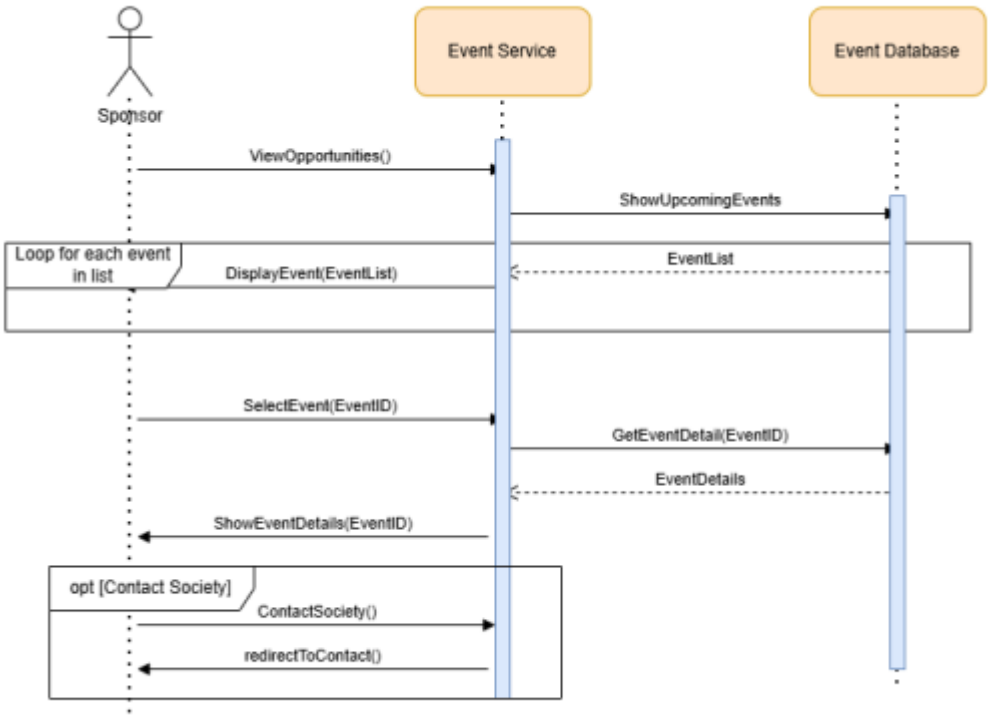


Figure 6.7 – Sequence Diagram: View Sponsorship Opportunities

6.8 Approve Events

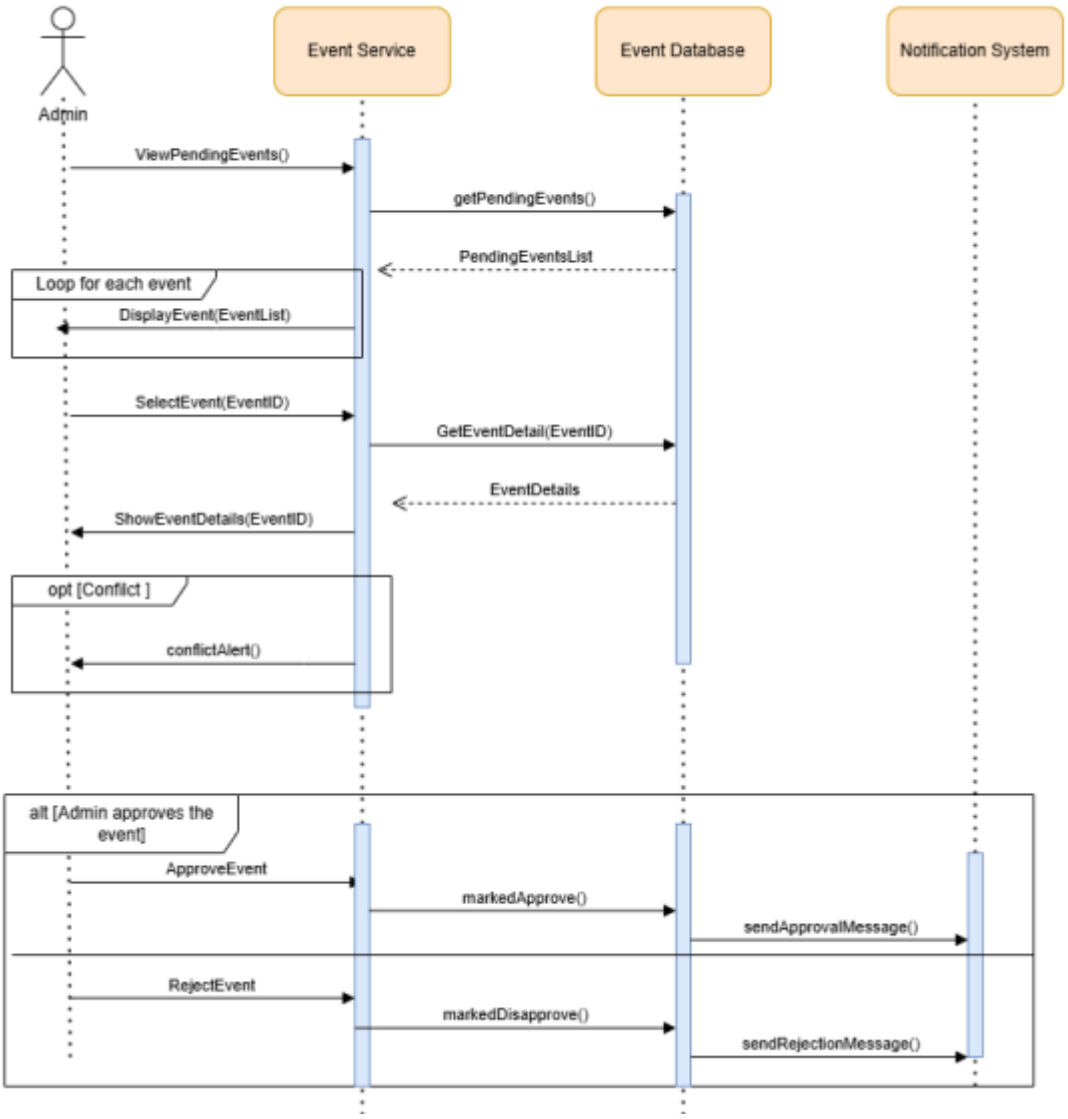
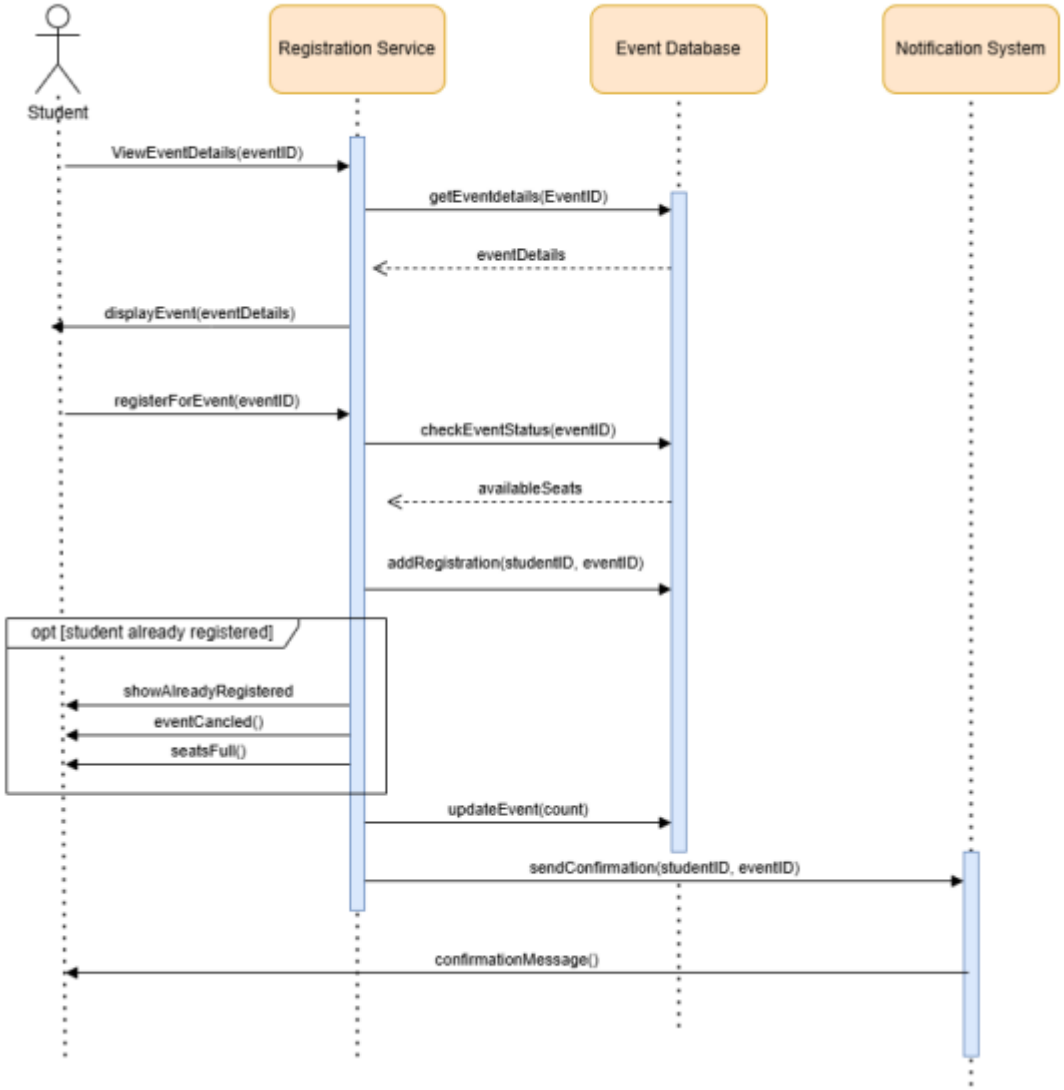


Figure 6.8 – Sequence Diagram: Approve Events

6.9 Register for an Event



6.9 – Sequence Diagram: Register for an Event

6.10 Create Event

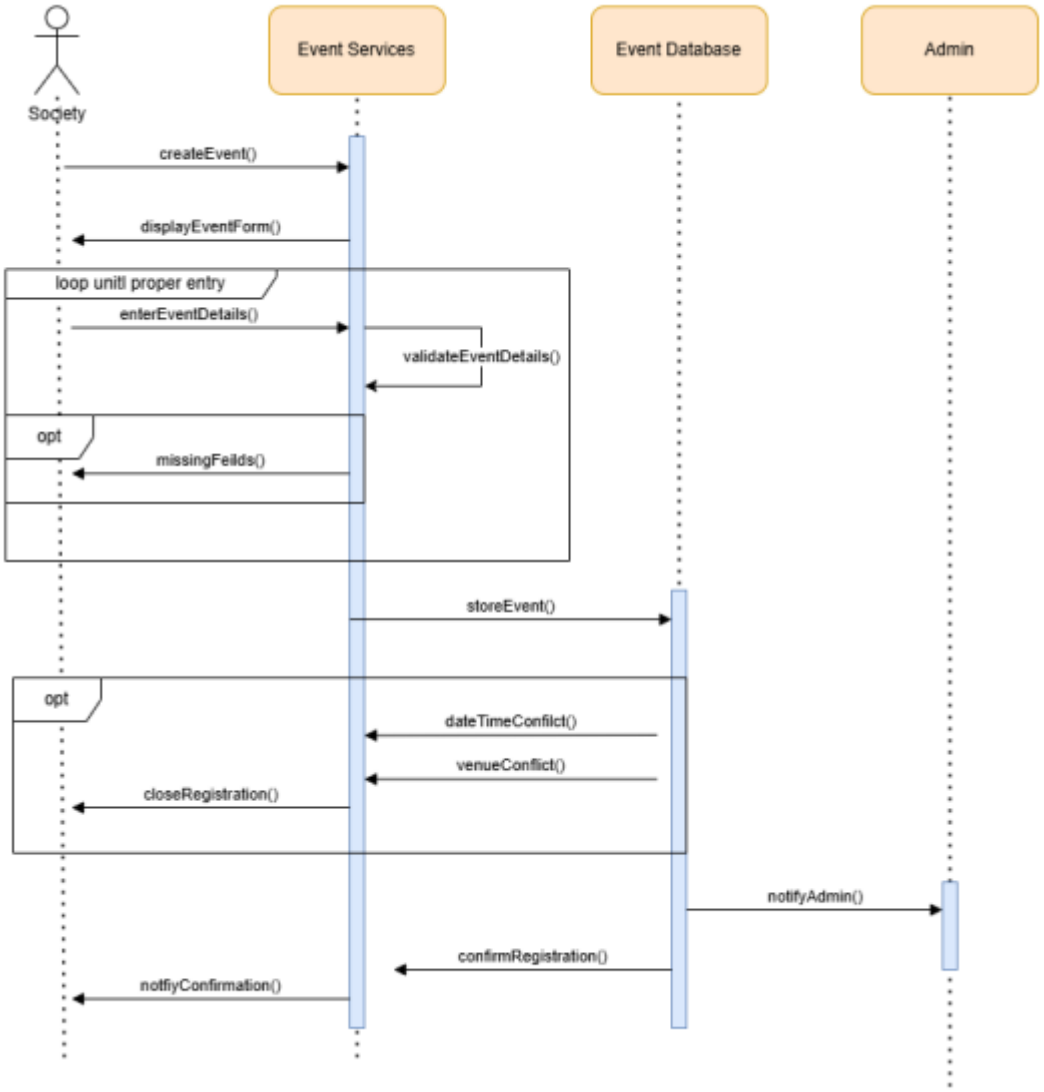


Figure 6.10 – Sequence Diagram: Create Event

6.11 Contact Society

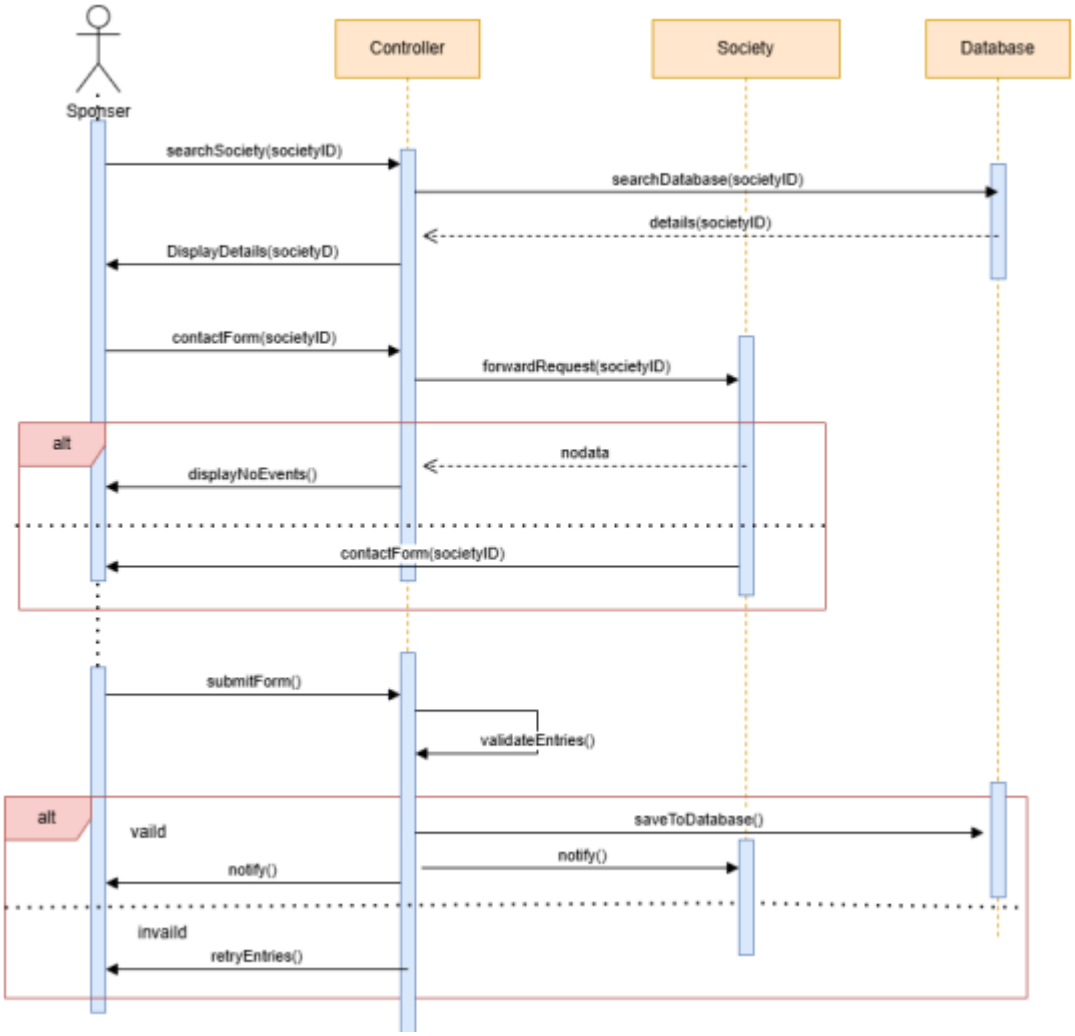


Figure 6.11 – Sequence Diagram: Contact Society

6.12 Manage Societies

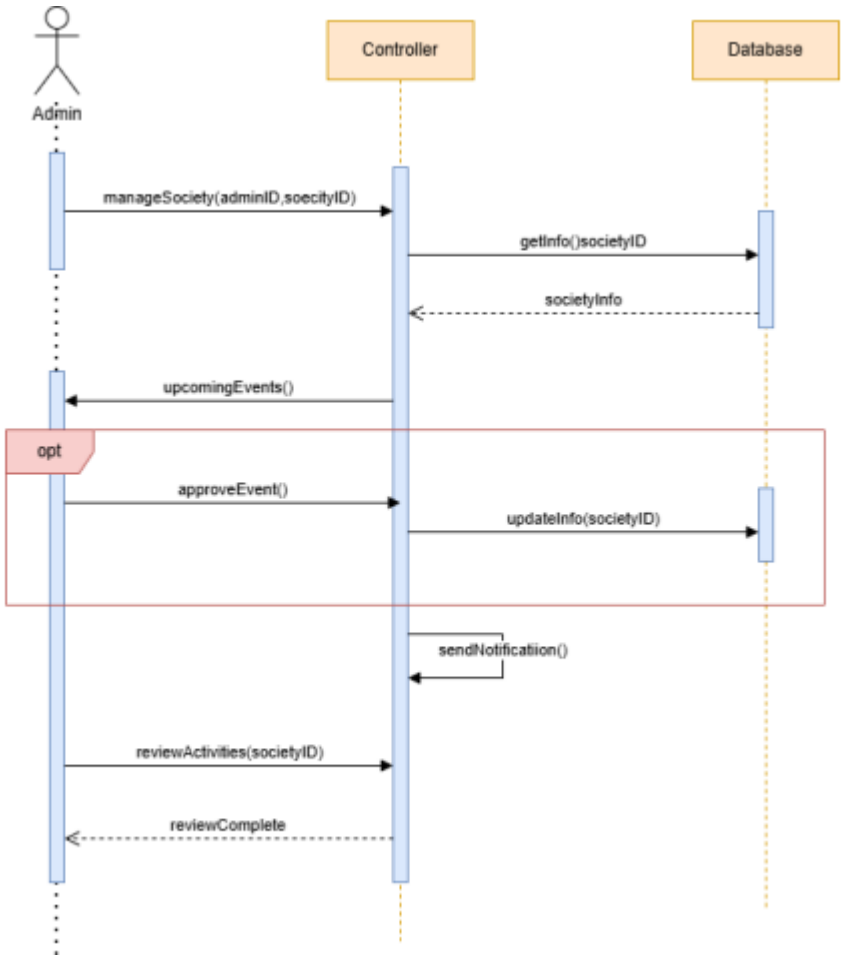


Figure 6.12 – Sequence Diagram: Manage Societies

7. Class Diagram

The Class Diagram models the static structure of UniVerse. The design applies GRASP principles throughout:

- **Information Expert:** Each class holds and manages its own data. For example, the Event class contains isOpenForRegistration(), hasSeatsAvailable(), and getDeadlineLabel() all computed from its own attributes.
- **Creator:** Society creates Event objects (createEvent method on the Society model). DashboardController creates EventRegistration and SponsorshipDeal objects at the point where user interaction triggers them.
- **Controller:** DashboardController mediates between all UI interactions and the service layer. LandingController handles landing-page interactions and delegates authentication to UserService.

- **Pure Fabrication:** DBConnection and Session have no real-world counterpart they are invented purely to manage infrastructure concerns (database connectivity and login state) without polluting domain classes.
- **Low Coupling / High Cohesion:** Service classes (EventService, RegistrationService, NotificationService, etc.) are independent of each other and of the UI. They can be tested and maintained in isolation.

The main classes are: abstract User (extended by Student, Society, Sponsor, Admin), Event, EventRegistration, Announcement, SponsorshipDeal, Notification, Session (singleton), DBConnection (singleton), and the controller classes.

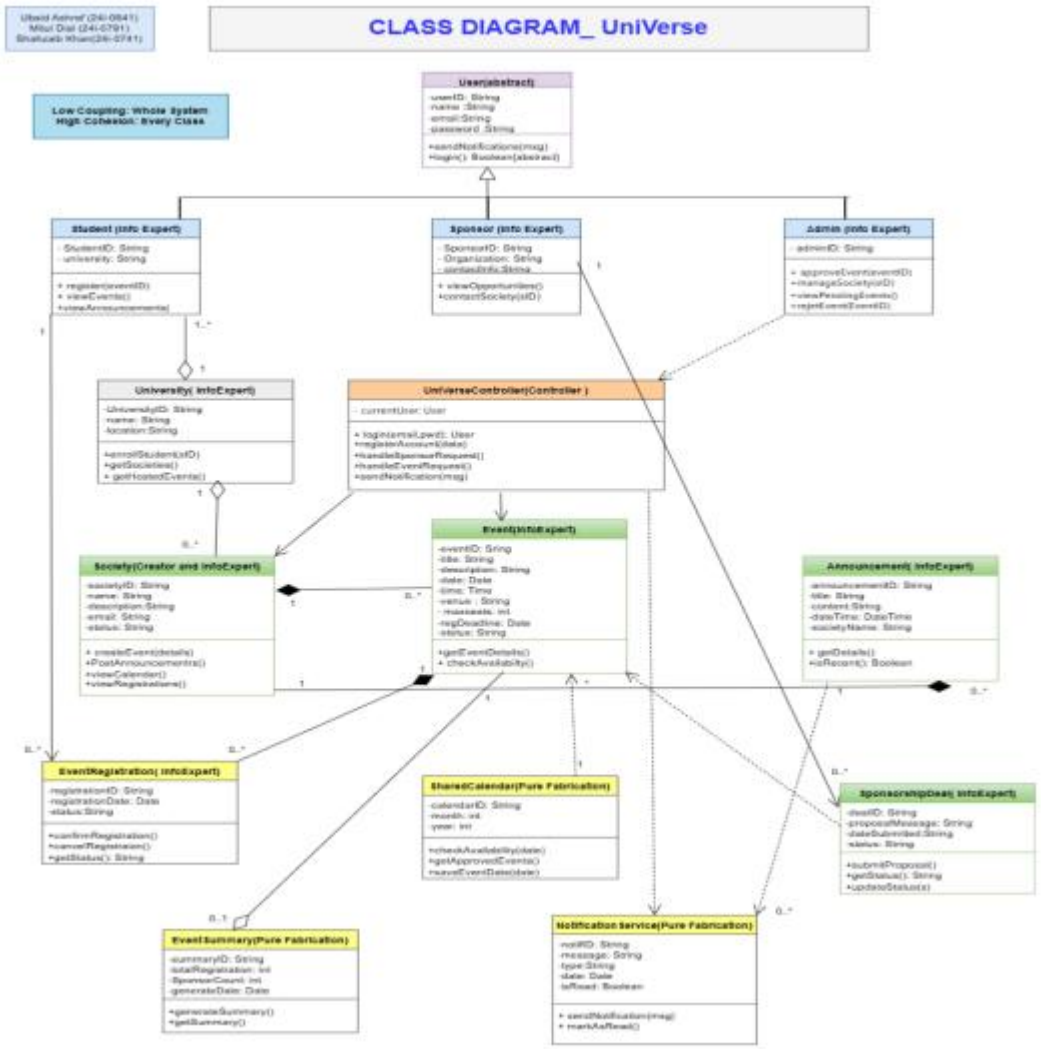


Figure 7.1 – UniVerse Class Diagram

8. Package Diagram

The Package Diagram organizes the UniVerse codebase into logical packages, showing the dependency direction between them.

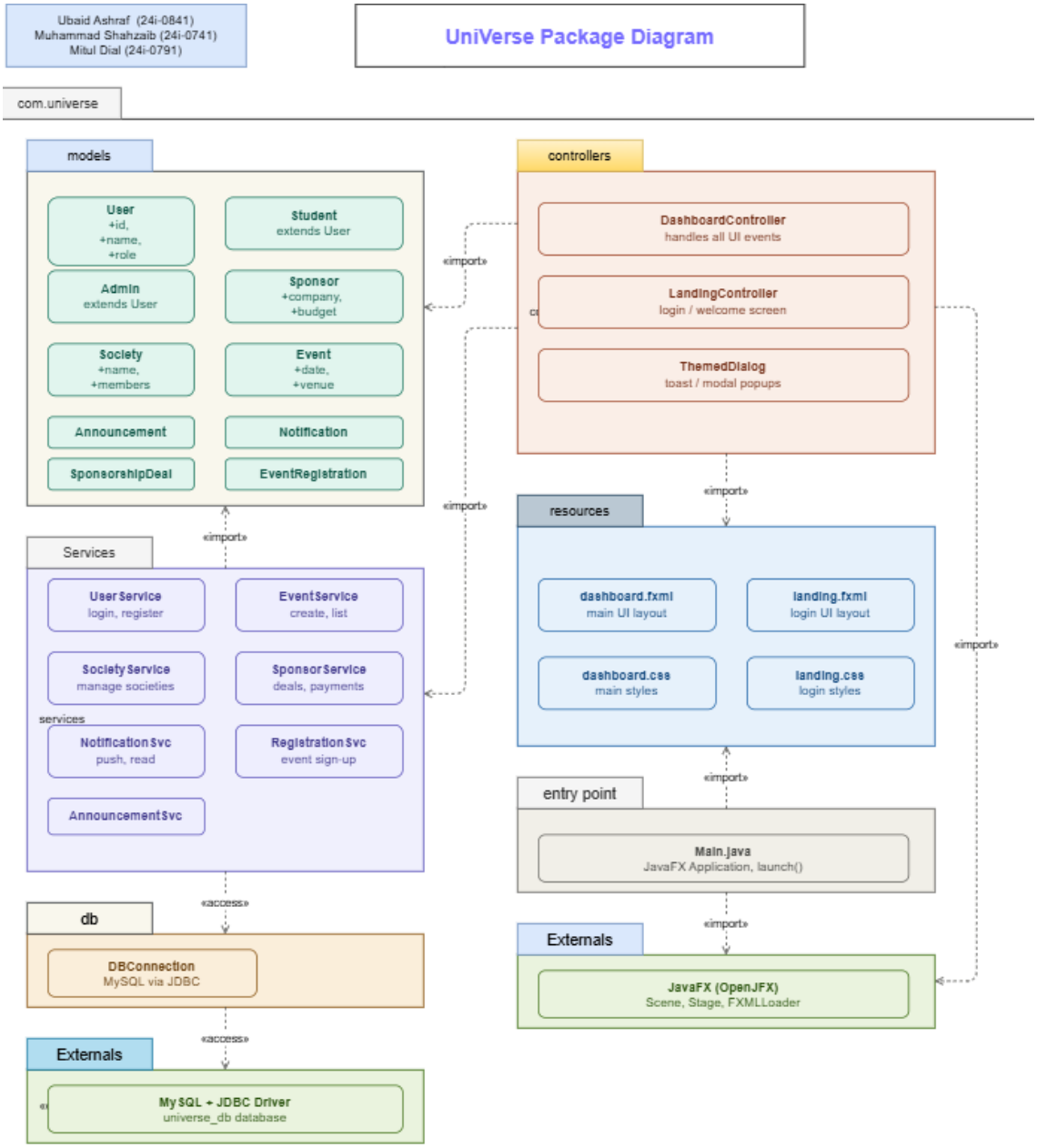


Figure 8.1 – UniVerse Package Diagram